

CAPSTONE PROJECT - MEETING MINUTES

The Alchemist: AI-Powered Mineral Data Explorer

Project Name:	The Alchemist - AI-Powered Mineral Data Explorer
Professor:	Xiaogang (Marshall) Ma (max@uidaho.edu)
TA:	Zhang, Jiyin (jiyinz@uidaho.edu)
Team Members:	Pratik Rauniyar, Sikha Chaudhary, Safal Shrestha, Bidhi Poudel

Meeting 1: Project Kickoff & Requirements Analysis

Date: Sep 15, 2025 | Time: 2025-09-15 3:30 PM

Problems/Issues Identified

N/A (Initial meeting)

Checkpoint/Deliverables

Project scope defined, technology stack selected (FastAPI, React, LangChain, Azure OpenAI, Mindat API), team roles assigned

Topics to Discuss

1. Understand project objectives and mineral data domain
2. Confirm technology selections (FastAPI, LangGraph, LangChain)
3. Discuss authentication approach (Supabase)
4. Review deployment platforms (Render, Vercel)
5. Establish GitHub branching strategy
6. Define initial project timeline

Results/Conclusion

Decision made to use LangGraph-based multi-agent architecture. Confirmed Supabase for authentication and PostgreSQL database. Established separate branches for each team member.

Meeting 2: System Architecture & Database Design

Date: Oct 6, 2025 | Time: 2025-10-06 3:30 PM

Problems/Issues Identified

Database schema design complexity, determining optimal table structure for sessions/messages, handling user authentication flow

Checkpoint/Deliverables

Database schema finalized with tables for Users, Profiles, Sessions, Messages, AgentOutputs. Supabase Auth integration configured. ER diagram completed.

Topics to Discuss

1. Review detailed system architecture (Frontend-Backend-Cloud Services)
2. Design and validate PostgreSQL schema
3. Plan Supabase Auth OAuth integration
4. Discuss data relationships (Users→Sessions→Messages)
5. Define API endpoint structure
6. Plan CORS configuration for frontend-backend communication

Results/Conclusion

Database design approved. Supabase project created and configured with proper tables and relationships. Authentication flow documented.

Meeting 3: FastAPI Backend Foundation Setup

Date: Nov 7, 2025 | Time: 2025-11-07 2:00 PM

Problems/Issues Identified

Setting up FastAPI project structure, managing dependencies, configuring Supabase connection strings

Checkpoint/Deliverables

FastAPI application structure created with proper routing, middleware setup (CORS), and database connection established. Health check endpoint working.

Topics to Discuss

1. Review FastAPI project structure and code organization
2. Verify all dependencies are correctly installed
3. Test database connection and SQLAlchemy ORM setup
4. Implement basic auth endpoints (register/login)
5. Create session/message CRUD endpoints
6. Test all endpoints with Postman/cURL

Results/Conclusion

FastAPI server running successfully with all basic endpoints functional. Database migrations working properly. Ready to integrate LangChain agents.

Meeting 4: LangGraph Multi-Agent System Implementation

Date: Nov 10, 2025 | Time: 2025-11-10 3:00 PM

Problems/Issues Identified

Implementing LangGraph state management, routing logic in Supervisor agent, ensuring proper agent communication

Checkpoint/Deliverables

Supervisor agent created with routing to 5 specialized agents (General, Geomaterial, Locality, Plot Generator, Tools Executor). Agent state machine working.

Topics to Discuss

1. Review LangGraph workflow and state management
2. Implement Supervisor node with intent classification
3. Create General Agent (greetings/FAQs)
4. Create Geomaterial Collector Agent (mineral data queries)
5. Create Locality Collector Agent (geographic queries)
6. Create Vega Plot Generator Agent
7. Test agent routing with sample queries

Results/Conclusion

All agents implemented and tested. Supervisor correctly routes queries to appropriate agents. Agent responses returning to frontend successfully.

Meeting 5: MCP Server & Tool Implementation

Date: Feb 2, 2026 | Time: 2026-02-02 3:30 PM

Problems/Issues Identified

Setting up FastMCP server, creating tool definitions, handling Mindat API integration, tool invocation in agents

Checkpoint/Deliverables

MCP server running on port 8010 with all 5 tools registered: collect_geomaterials, collect_localities, profile_sample_data, and visualization tools. Tools tested with Mindat API.

Topics to Discuss

1. Set up FastMCP server and verify it's running
2. Implement collect_geomaterials() tool with Mindat API integration
3. Implement collect_localities() tool
4. Implement data profiling tool for analytics
5. Test tool invocation from agents
6. Handle tool responses and error cases
7. Configure MCP_SERVER_URL for local/production environments

Results/Conclusion

All tools successfully integrated and tested. MCP server communicating properly with agents. Mindat API queries returning expected mineral data.

Meeting 6: React Frontend Development

Date: Apr 2, 2026 | Time: 2026-04-02 3:30 PM

Problems/Issues Identified

Setting up Vite React project, implementing authentication UI, creating chat interface components, styling with Bootstrap

Checkpoint/Deliverables

React/Vite project structure established with all components created: SignIn, Signup, ChatWindow, MessageBubble, VegaChart, Sidebar. Responsive dark theme applied.

Topics to Discuss

1. Set up Vite React project with React Router
2. Implement authentication pages (SignIn/Signup) with Supabase
3. Create chat interface main page
4. Build MessageBubble component for message display
5. Implement ChatInput component with API integration
6. Create Sidebar for chat history/navigation
7. Build VegaChart component for visualization rendering
8. Implement settings and profile modals

Results/Conclusion

All frontend components completed and styled. Authentication flow working with Supabase. Chat interface fully functional and connected to backend API.

Meeting 7: Full Stack Integration & Testing

Date: May 7, 2026 | Time: 2026-05-07 4:00 PM

Problems/Issues Identified

End-to-end testing, debugging API communication, handling edge cases, performance optimization

Checkpoint/Deliverables

Complete user flow tested: Login → Chat Creation → Query → Agent Processing → Result Display. All components communicating properly. Basic load testing passed.

Topics to Discuss

1. Test complete user journeys (login to visualization)
2. Test all query types (mineral data, locality, visualization)
3. Verify error handling and user feedback
4. Test session persistence and chat history
5. Performance testing with multiple concurrent requests
6. Mobile responsiveness testing
7. Security testing (auth tokens, CORS, input validation)

Results/Conclusion

All integration tests passed. System is stable and ready for deployment. Minor UI/UX improvements identified and implemented.

Meeting 8: Backend & Frontend Deployment Configuration

Date: Jun 4, 2026 | Time: 2026-06-04 4:00 PM

Problems/Issues Identified

Configuring Render.com and Vercel for production, managing environment variables, setting up CI/CD

Checkpoint/Deliverables

Backend deployed on Render with MCP server running separately. Frontend deployed on Vercel. Both connected and communicating in production environment.

Topics to Discuss

1. Prepare .env files for production (without exposing secrets)
2. Configure Render.com services (Backend + MCP)
3. Set up environment variables in Render dashboard
4. Deploy backend and verify health checks
5. Configure Vercel for frontend deployment
6. Set environment variables for production in Vercel
7. Test production URLs and API connectivity
8. Configure custom domain if applicable

Results/Conclusion

Both services successfully deployed and live. Production URLs verified working. All API calls from frontend to production backend successful.

Meeting 9: Performance Optimization & Final Presentation Prep for Capstone Expo

Date: Jul 2, 2026 | Time: 2026-07-02 4:00 PM

Problems/Issues Identified

Optimizing agent response times, improving UI/UX, preparing presentation materials, documenting project

Checkpoint/Deliverables

LangSmith monitoring integrated for agent tracing. UI optimized for faster load times. Comprehensive documentation and presentation deck prepared.

Topics to Discuss

1. Review LangSmith traces for agent performance
2. Optimize LLM prompts for faster responses
3. Implement response caching where appropriate
4. Refine UI/UX based on user feedback
5. Complete comprehensive project documentation
6. Prepare presentation slides and demo
7. Create user guide and deployment documentation
8. Plan final testing and QA checklist

Results/Conclusion

Project optimized and ready for final presentation. All documentation complete. Demo prepared showcasing key features and agent capabilities.